

Mizar Evo: Design Principles

Connecting automatic proof, readable mathematics, and checked computation
TPP 2026, RIKEN AIP Tokyo Office, November 2026

Mizar Evo project

TPP 2026, RIKEN AIP Tokyo Office, November 16–17, 2026

Level	Meaning	Marked how
fact	existing systems, published benchmarks, the Mizar Evo specification and main branch	no tag
research hypothesis	a claim that Mizar Evo is built to test	tagged in the text
future direction	a goal with no fixed date or design yet	tagged in the text

Code labels follow the Bialystok deck: exact MML excerpt, specification example, sketch.

- The specification and the implementation are different things. One frame near the end separates them.

0.3 - Current Mizar: Six Challenges And Design Principles

Challenge in modernizing Mizar	New design principle
§1 Foundation: retain logic and MML; renew tools	First-order logic, set theory, and a small kernel
§2 Writing: implicit types, registrations, overloads	Mathematical abstraction; explicit, traceable choices
§3 Generics: separate definition and scheme mechanisms	Templates for definitions, theorems, and schemes
§4 Computation: connect proofs and procedures	Contracts, invariants, and termination checks
§5 Development: tools for a large library	Modules, namespaces, packages, incremental builds, LSP
§6 Checking: external search and evidence	ATP flow; derive formula instances; trusted SAT check

1.1 - Keep The Foundation, Rebuild The Tools

*Keep the mathematical layer.
Rebuild the tools under it and around it.*

- Mizar Evo keeps first-order logic and Tarski-Grothendieck set theory as the base logic.
- It keeps soft types, modes, attributes, registrations, structures, and declarative proofs.
- **Separate proof search from checking; a small kernel decides acceptance.**
Details in section 6.

1.2 - Current Mizar: Functions In Set Theory

Function application is a defined set-theoretic relation **exact MML excerpt**

```
func f.x -> set means  
:Def2:  
[x,it] in f if x in dom f otherwise it = {};
```

A typed function is a soft type over partial functions **exact MML excerpt**

```
definition  
  let X,Y;  
  mode Function of X,Y is quasi_total PartFunc of X,Y;  
end;
```

- Here a function is an ordinary first-order object: a set of pairs. Application is defined, not built into the logic.
- Nothing in the base logic has to be higher-order.
- The price: if you write raw set theory, the domain, the graph, functionality, and membership all appear in the text.

2.1 - Preserve And Extend Mathematical Writing

What the author actually writes **sketch** (valid in current Mizar and Mizar Evo)

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- It is the same set-theoretic function. But the author writes Function of X,Y and f.x, not pairs and domains.
- Soft types, modes, attributes, registrations, schemes, and declarative proofs are not just a nicer way to write the same thing. They are the language design that lifts first-order set theory up to readable mathematics.
- New specification: preserve and extend mathematical writing; make implicit type, registration, and overload choices explicit and traceable.

2.2 - Registrations: Give Automatic Chains Names

Named automatic rules **specification example**

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;
  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

Starting fact	Automatically derived fact	Recorded rule
S is empty	S is finite	EmptyImpliesFinite
S is finite	S is countable	FiniteImpliesCountable

- **Keep automatic chaining; require a label on every registration item and record the applied path.**
- Labels explain which rule a proof depends on. Explicit by citations remain optional for automatic application.

2.3 - Structures: Stored Fields And Derived Properties

Data and canonical values have different roles **specification example**

```
definition
  struct AddLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    property zero -> Element of carrier;
  end;
end;
```

- **Fields store data and are constructor arguments. Properties get a uniquely determined value from a separate implementation.**
- The declaration of zero supplies its type. A means implementation proves existence and uniqueness; an equals implementation supplies a term.

2.3a - Inheritance Can Be Declared Later

After declaring AddLoopStr, add a parent mapping **specification example**

```
definition
  inherit AddLoopStr extends LoopStr where
    field carrier from carrier;
    field add from binop;
    property zero from unit;
  end;
end;
```

- **Separate the structure declaration from later inheritance declarations, much like implementing a Rust trait after defining a type.**
- from renames the parent roles: binop becomes add, and unit becomes zero.
- Each declaration has one parent. Identical member types need no proof; narrower types need a coherence proof.

2.3b - Checked Diamonds And Reused Group Theorems

Two paths in the same hierarchy **sketch**

```
AddLoopStr -> LoopStr -> Magma
AddLoopStr -> AddMagma -> Magma
```

A Group theorem applied to a ring's additive view **sketch**

```
definition
  let T be type extends Group;
  theorem RightUnit[T]:
    for x being Element of T.carrier holds T.binop(x,T.unit) = x
  proof ... end;
end;
RightUnit[R qua AddLoopStr]  :: gives R.add(x,R.zero) = x
```

- Track member roots and inheritance paths; check shared members while keeping the operation views distinct.
- Ring's additive view is a Group. Reuse its theorem through renamed members; multiplication is only required to be a monoid.

3.1 - Current MML: Result Types For The Sum Of Functions

Example: the sum of functions. Add their values at each point.

Result-type registrations (sketch, separate complex-valued and real-valued input blocks):

```
cluster f1 + f2 -> complex-valued;  
cluster f1 + f2 -> real-valued;
```

Step in VALUED_1	Type information
Define pointwise addition once	The result is a Function
Refine the codomain	Separate complex and real PartFunc redefinitions
Recover the full input domain	Separate totality registrations

- **The mathematical operation is already shared. The result-type refinements still follow the value types.**

3.1a - Template: One Body And Concrete Result Types

Generic functor **sketch** (existence and uniqueness proofs omitted)

definition

```
let T be type extends non empty AddMagma;  
let I be non empty set;  
let f, g be Function of I, T.carrier;  
func AddDef: Add[T,I](f,g) -> Function of I,T.carrier means  
  for i being Element of I holds it.i = T.add(f.i,g.i);  
synonym f +[T,I] g for Add[T,I](f,g);  
end;
```

Call (real R, complex C)¹

Result type

f + g

Function of I,R.carrier

u + v

Function of I,C.carrier

- A synonym gives infix +; infer template arguments from declared types.

¹Explicit: f +[R,I] g, u +[C,I] v. Omit arguments only when declared types determine them uniquely. Write qua for ambiguous inheritance paths.

Induction as a theorem with a predicate parameter specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- A predicate parameter gives a family of first-order theorems, one for each predicate you put in.
- Instantiation is explicit: by `NatInduction[P]`, `Base`, `Step` after a `defpred`.
- Functor parameters are schema-level symbols, not sets. This keeps the logic first-order.
- Details: Bialystok deck, Story 6 (`PermProduct [T]`, `qua views`), and Backup 4 here.

4.1 - Computation: Verified Algorithms

Hoare-style checking of readable procedures:

$$\{ \textit{requires} \} \textit{algorithm body} \{ \textit{ensures} \}$$

- Use familiar pseudocode: assignment, if/else, while, for, and return. Contracts and loop invariants state what must hold.
- Generate first-order verification conditions using Hoare logic. Verify correctness and, when required, termination.
- Verify algorithms themselves and use verified procedures to support proofs.

4.2 - Algorithms: Contracts, Proofs, Computation

Euclid's algorithm with a contract **specification example**

```
terminating algorithm EuclidGcdDef: euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a; var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y); decreasing y;
    const r := x mod y; x := y; y := r;
  end;
  return x;
end;
```

- terminating requires termination for every input satisfying requires. The verifier checks the invariant and strict decrease of y .¹
- After verification, the algorithm is promoted to a mathematical functor, usable in formulas and proofs.

¹Loop annotations resemble Dafny's invariant / decreases; Evo uses invariant / decreasing.

4.2a - Termination And Functor Promotion

Algorithm form	Meaning under requires
Without terminating	If the call returns, its contract holds
Verified terminating	Totality under requires; usable as a functor

Reason about Euclid through its contract **sketch**

```
let a, b be Nat;  
assume a >= 1 & b >= 1;  
thus euclid_gcd(a,b) = Gcd(a,b) by EuclidGcdDef;
```

- **Verified definitional algorithms also give defining equations. Euclid has a loop: use only its totality and contract axioms.**
- by computation is specified separately; MVM execution and code extraction remain future work.

4.3 - Euclid: What Must Be Proved?

One loop step: old state (x, y) with $y > 0$; set $r = x \bmod y$; new state $(x', y') = (y, r)$.

Obligation	Mathematical reason
Establish the invariant	$x=a, y=b$; the precondition gives positivity
Preserve the invariant	$\text{Gcd}(x, y) = \text{Gcd}(y, r), y \geq 1, r \geq 0$
Decrease the Nat measure	$y' = r$, and $0 \leq r < y$
Establish the postcondition on exit	$y=0; \text{Gcd}(a, b) = \text{Gcd}(x, 0) = x$

- **The library supplies the GCD and remainder lemmas. These obligations go through theorem checking; SAT alone does not know arithmetic.**
- Replacing the final $y := r$ with $y := x$ loses strict decrease. This is a specification walkthrough.

5.1 - Environment: What Must An Author Import?

Current Mizar environment, shortened from ALGSTR 0 [sketch](#)

```
environ
vocabularies ... STRUCT_0 ...;
notations ... STRUCT_0;
constructors ... STRUCT_0 ...;
registrations ... STRUCT_0;
theorems STRUCT_0;
```

List	What it supplies
vocabularies / notations	Symbols / notation
constructors	Constructors
registrations / theorems	Automatic type facts / cited theorems

- Choose articles by role; notations and definitions also depend on article order.

5.1a - Imports And Dependency Tracking

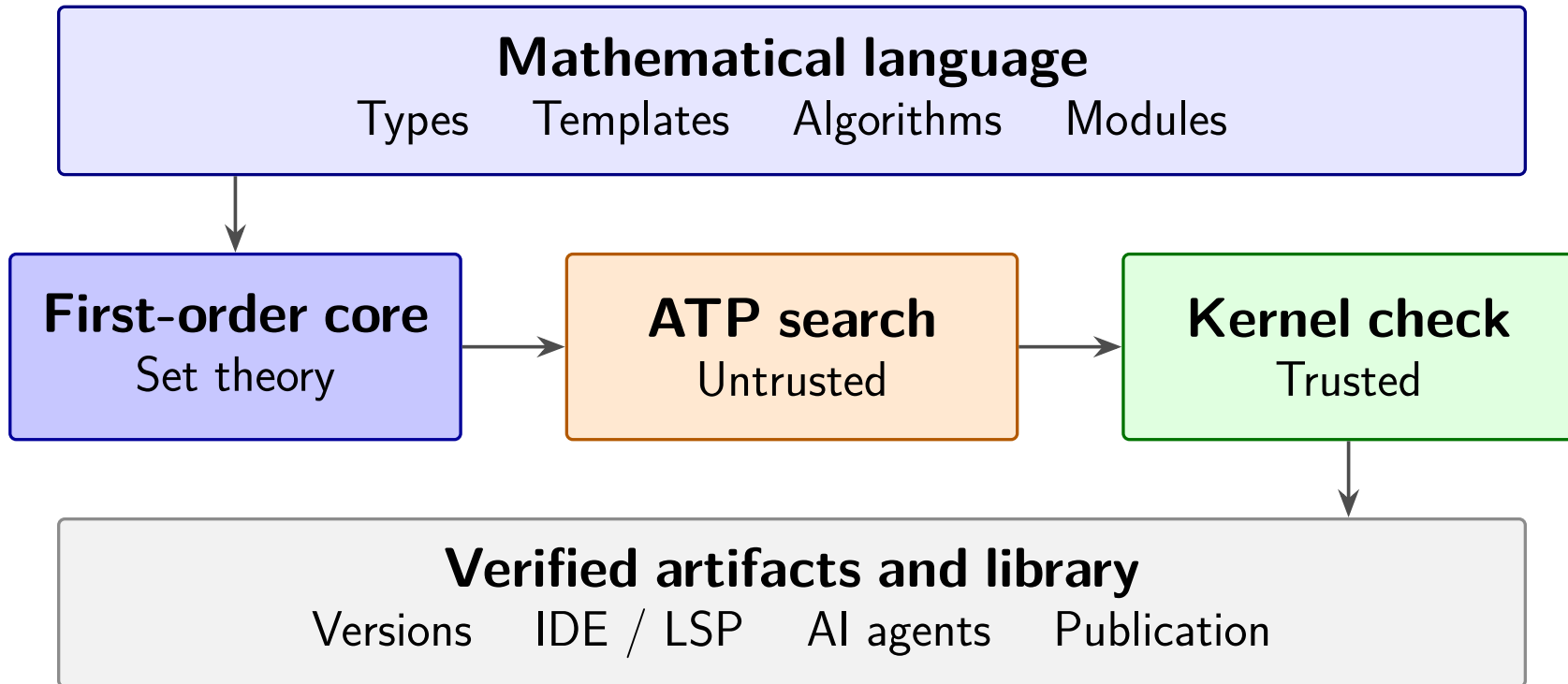
New specification: module imports **specification example**

```
import .function;  
import mml.algebra.structure.sorted;
```

Stage	What the author can inspect
Import	The module's public definitions, theorems, and registrations
Resolve	Source fully-qualified names and registration traces
Reuse	Validated dependency fingerprints for incremental builds

- **Bring public items in together, then track what was actually used.**
- Packages and lock files fix dependency versions; IDE diagnostics can expose the resolved names and traces.

5.2 - The Whole Picture



- Rich mathematics, a small first-order core, and modern infrastructure.

6.1 - Automation: Separate Search From Checking

Isabelle/HOL: Sledgehammer suggests a locally checked proof **sketch**

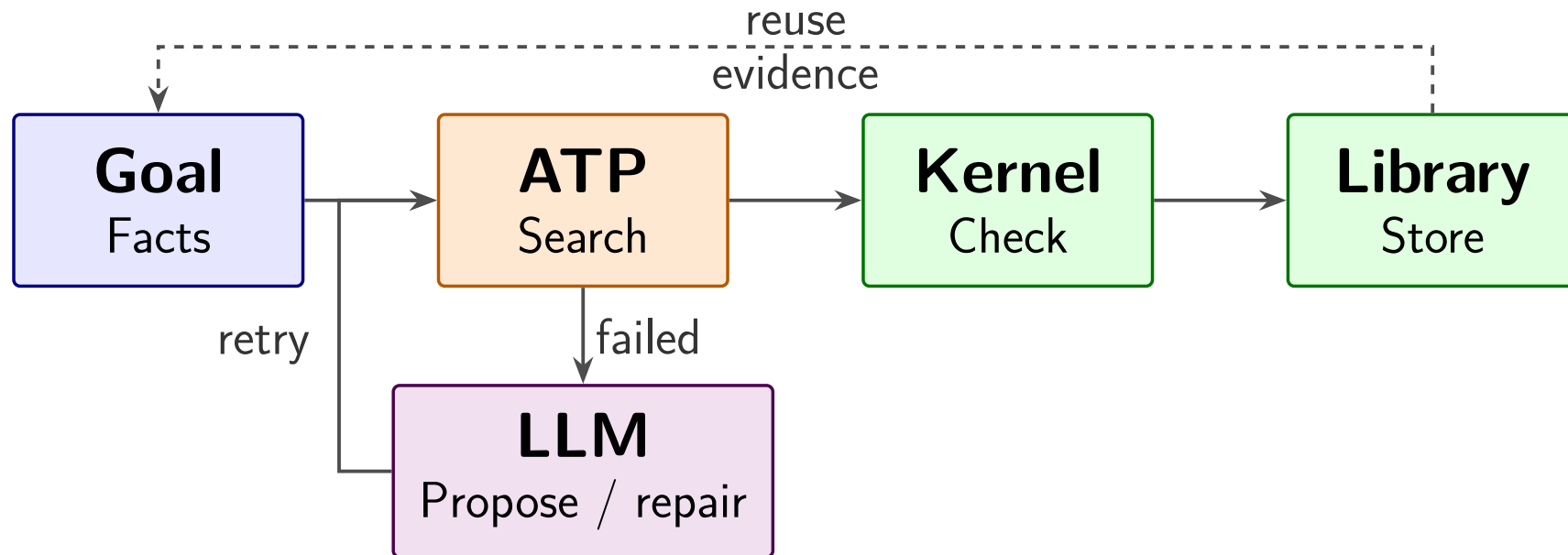
```
have "Q a"  
  sledgehammer  
  by (metis allPQ pa)
```

Mizar Evo: cited facts justify a declarative step **sketch**

```
assume AllPQ: for x being object holds P(x) implies Q(x);  
assume Pa: P(a);  
thus Q(a) by AllPQ, Pa;
```

- **Shared pattern: goal and facts → external search → local trusted acceptance. A prover's success report alone is insufficient.**
- Isabelle reconstructs a proof, often by local Metis reproof. Mizar Evo checks formula/substitution evidence through instantiation and SAT.

6.2 - Using ATPs: Search, Check, And Reuse



- Goal and facts → ATP search → kernel check → store in the library and reuse.
- For open goals, revise lemmas or the plan and retry. The figure shows design intent.

6.3 - Resolution Tree And Extracted Substitutions

Candidate extraction from a Resolution log **sketch**

F1: forall x. (P(x) implies Q(x)); F2: forall y. (Q(y) implies R(y))

H: P(a); goal: R(a); refute: F1 & F2 & H & not R(a)

```
{not P(x), Q(x)}           {not Q(y), R(y)}
  \                         /
  sigma1 = {x := y}         :: unify Q(x), Q(y)
  {not P(y), R(y)}         {P(a)}
      \                     /
      sigma2 = {y := a} :: unify P(y), P(a)
      {R(a)}             {not R(a)}
          \               /
          sigma3 = {}
              {}
```

- The untrusted extractor composes substitutions along the branches: save $x := a$ for F1 and $y := a$ for F2.

6.3a - Saved Evidence To Concrete SAT Clauses

Saved candidate, kernel preprocessing, and SAT input sketch

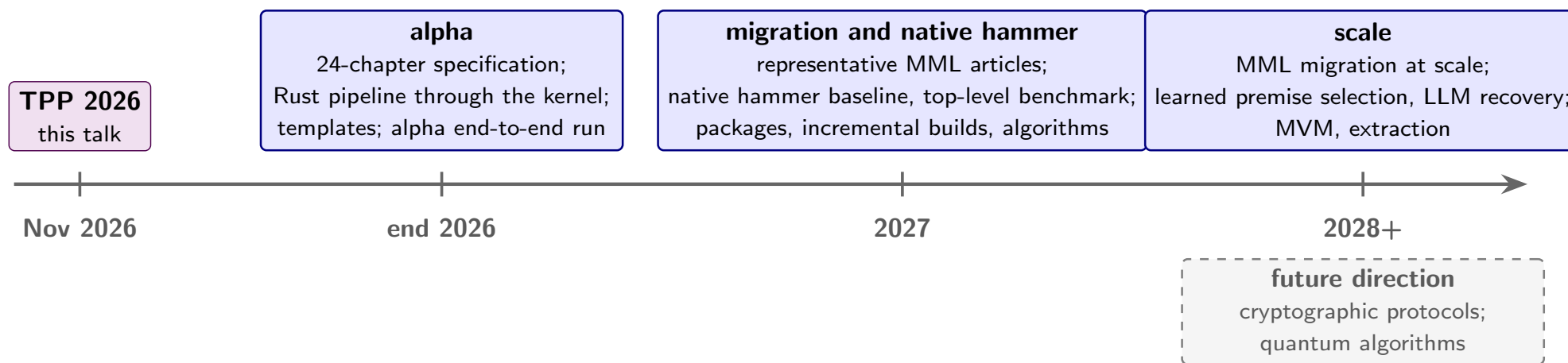
```
save: F1,F2,H (source bindings); F1[x:=a], F2[y:=a]; goal R(a), refute
check: source/context, binders, capture avoidance
instances: P(a) implies Q(a); Q(a) implies R(a); P(a); not R(a)
atoms: p=P(a)=1, q=Q(a)=2, r=R(a)=3
logical CNF: (not p or q) & (not q or r) & p & not r
Tseitin: s=4 iff (not p or q); t=5 iff (not q or r)
SAT clauses (DIMACS example; 0 ends each clause):
p cnf 5 10
1 0          -3 0
1 4 0        -2 4 0      -1 2 -4 0      4 0
2 5 0        -3 5 0      -2 3 -5 0      5 0
```

- The small kernel derives instances and CNF from checked evidence. Its SAT checker finds UNSAT, so accept $R(a)$; external Resolution steps are not replayed.

Where The Project Stands (October 2026)

- **Specification:** 24 chapters plus appendices. English is the canonical language.
- **Implemented on the main branch:** the Rust frontend (lexer, parser, syntax tree); name resolution and type checking on an alpha corpus; proof-obligation generation with deterministic discharge; ATP problem encoding and candidate evidence; SAT-backed kernel evidence checking; cache, fingerprint, and build-scheduling milestones.
- **In progress:** end-to-end integration from source to verified artifact; the LSP server; the documentation generator.
- **Planned for later:** MVM execution, code extraction, library-wide premise selection, MML migration.
- This talk does not claim any end-to-end results with external provers.

Roadmap



- **2026**: finish the specification, the Rust pipeline through the kernel, template processing, and an alpha end-to-end run.
- **2027**: migrate representative MML articles, build a native hammer baseline, and benchmark on top-level theorems, in the same units MizAR uses.
- **2028 and later**: scale up the migration; learned premise selection; LLM-assisted failure recovery; MVM and extraction. Cryptographic and quantum targets stay future directions.

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- Thank you. I welcome your questions, especially on MML migration, usability, and how to evaluate automation.

Backup 1. MizAR 60 In Detail

Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

- Dataset: MML 1147 exported by MPTP, 57,897 theorems including unnamed top-level lemmas; the same version as the MizAR 40 evaluation, so the results can be compared.
- Holdout: 1,690 / 2,896 (58.4%), without user premise help, 420 CPU s. Training/development/holdout: 90:5:5 (MizAR 40: about 40.6%).
- Over 75% proved when the premises can be chosen from the library by a human or a machine (MizAR 40: 56%).
- Strongest single method: 40% in 30 s in hammering mode; 60% in 120 s with human premises.
- Transfer: the strongest method also works on 13,370 new theorems from 242 new articles in MML 1382.
- Methods: E and Vampire with ENIGMA and Deepire guidance, learned premise selection, and a loop that trains on millions of ATP proofs.

Backup 2. Sledgehammer Evaluations In Detail

Evaluation	Dataset and method	Success
ITP 2022	5,000 AFP goals, 16 greedy configurations	68.8%
Magnushammer (preprint 2023)	1,000 PISA theorems, Sledgehammer	38.3%
Same PISA evaluation	Magnushammer with learned premise selection	59.5%

- ITP 2022: 50 entries, 100 goals each; MePo, 512 facts in the base setting, 30 CPU seconds per configuration. Configurations are selected using the evaluation set. Reconstruction is not evaluated.
- PISA: Isabelle2021-1, Sledgehammer timeout 30 seconds, five local provers, union over several settings. Success requires a proof checked by Isabelle.
- Close dates do not make local goals and whole theorems, or search success and checked proofs, the same measure.

Backup 3. HOL-To-FOL Encodings And Reconstruction

- Function application: a variable function applied to an argument becomes $\text{app}(F, X)$; applications of constants may stay curried or be flattened.
- Lambda abstraction: lambda lifting introduces fresh constants with defining equations; combinator translation is the alternative.
- Types: polymorphic HOL types are encoded by guards, tags, or monomorphization; the choice affects soundness, completeness, and prover performance.
- Booleans: Boolean-valued terms inside formulas need a separate encoding.
- Reconstruction: `metis` with the used lemmas, `smt` replay of SMT proofs, or generated `Isar` text; evaluations count reconstruction failures separately.

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Backup 4. Template Instantiation: One Proof, Many Views

Bounded type parameter and generic theorem (specification example, proof omitted):

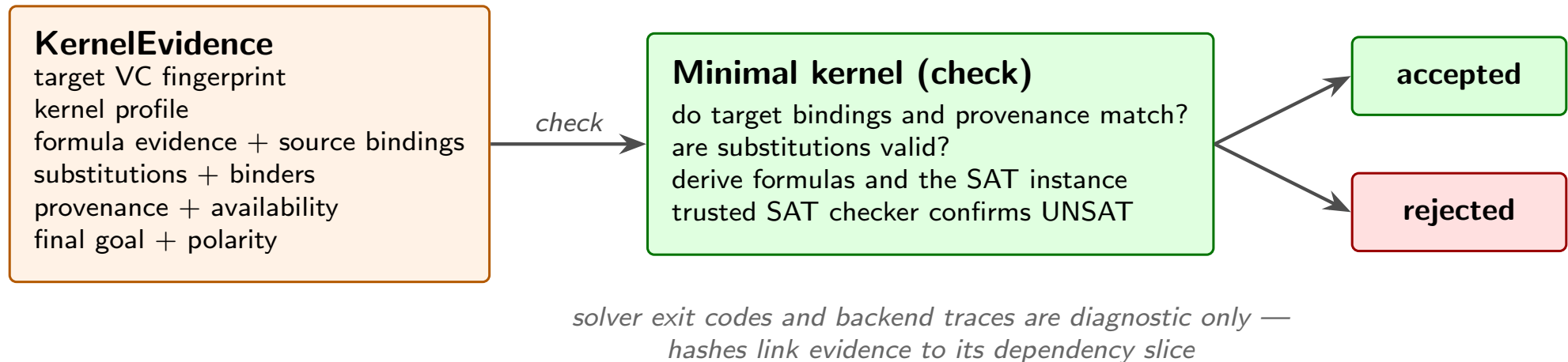
```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Instantiation **specification example** (with the required registrations)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma]  :: additive view
```

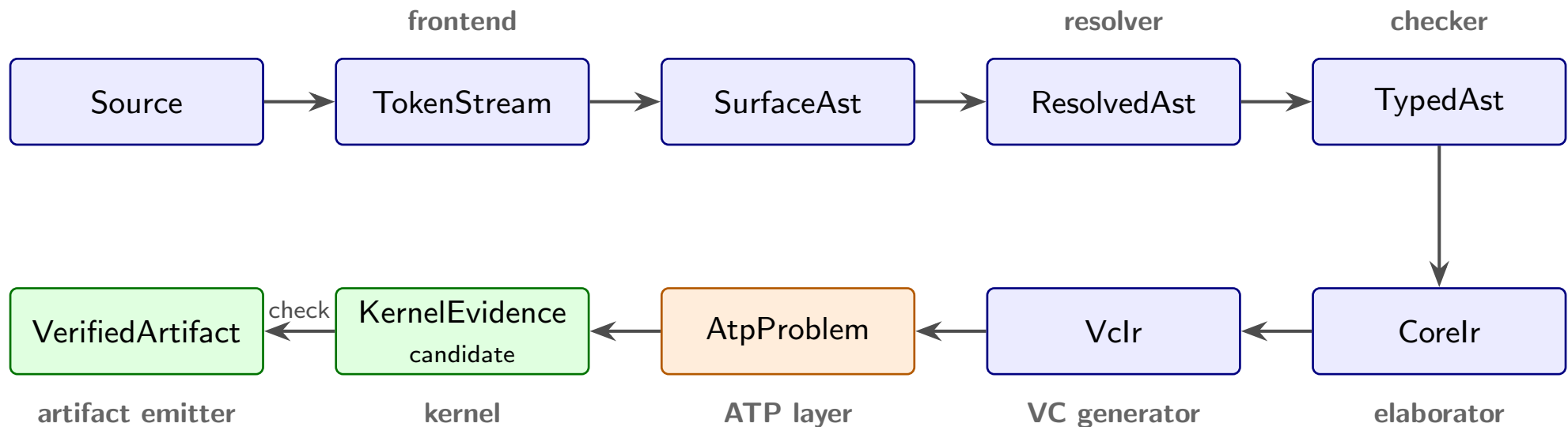
- A ring reaches Magma along two paths; qua selects the view, and the view also sets the notation.

Backup 5. Kernel Evidence And The Trusted SAT Check



- The evidence holds source formulas, explicit substitutions, provenance, and target and goal bindings. The kernel checks them, builds a deterministic SAT problem, and requires UNSAT from its trusted in-process SAT checker.
- Backend proof traces, SMT proof objects, logs, and exit codes are for diagnosis only.

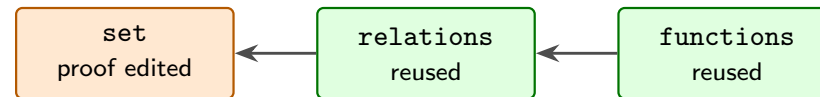
Backup 6. The Core ATP Path



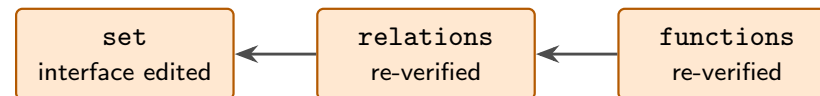
- Only obligations that are still open after deterministic discharge go to ATPs. The earlier discharge also needs replayable evidence.
- Each boundary states who owns a fact, which artifact records it, and what must be checked again after a change.

Backup 7. Incremental Verification

theorem proof edit:
public statement + accepted status unchanged — importers reuse



interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

- A proof-body edit does not rebuild importers when the exported statement and the accepted status stay the same. An interface change re-verifies the dependency cone.
- All relevant cache keys must match; missing data is a cache miss. Cache reuse is never proof authority: a clean build must reproduce every acceptance.

Backup 8. Package Manifest

Mizar Evo **specification example**

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

- Reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence.
- Versioned reuse replaces manual copying between article sets; an aggregate module can export a whole topic through one import.

Source: doc/spec/en/23.package_management_and_build_system.md; Bialystok deck, Story 1.

Backup 9. Forty-Five-Minute Additions

Add selected examples where they fit in the story:

- ① HOL-to-FOL function encoding worked through on one goal (app, partial application, lambda lifting).
- ② Mizar function representation: `Function of X,Y, f.x`, soft type as a set-theoretic object.
- ③ Template: `PermProduct [T]` with additive and multiplicative views (Backup 4).
- ④ Algorithm: Euclidean GCD plus `by computation` and the promotion to a functor.
- ⑤ Structures and views: `AddMagma`, `MulMagma`, `Magma` (Bialystok Story 2).
- ⑥ Modern infrastructure: `mizar.pkg`, namespaces, the fingerprint graph (Backups 7-8).
- ⑦ Checking: evidence instantiation and the SAT check (Frame 6.3; details in Backup 5).

Backup 10. Sources And Attribution

Exact MML excerpts used in this talk:

Purpose	Source	Lines	Used in
Function application f.x	funct_1.miz	138-140	Frame 1.2
Function of X,Y as a soft type	funct_2.miz	87-90	Frame 1.2

Source URLs:

- FUNCT_1: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- FUNCT_2: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz

Attribution note:

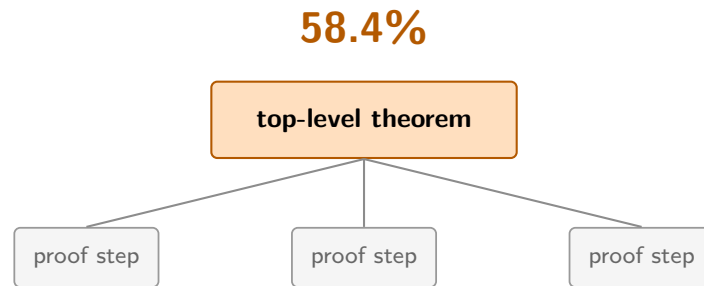
- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.
- Benchmark figures: see Backups 1-2 and references.bib. Verify bibliographic metadata against publishers before the final deck.

Backup 11. Comparing Evaluation Conditions

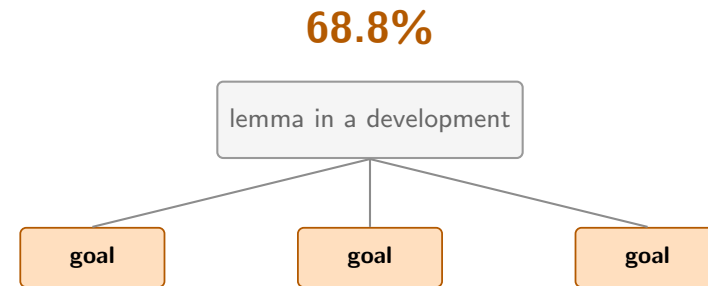
	MizAR 60 (ITP 2023)	Sledgehammer / AFP (ITP 2022)
success	58.4% (1,690 / 2,896)	68.8% (3,440 / 5,000)
counts	MML 1147: 2,896 holdout theorems/lemmas (57,897 total)	5,000 local goals; 50 AFP entries
premises	learned selection from the whole library	MePo; base 512 facts, varied by configuration
budget	portfolio: 420 CPU s	16 greedy configurations, 30 CPU s each: 480 CPU s
status	ATP proof found (hammering)	external proof found; no Isabelle reconstruction

- Different units, premises, budgets, and configurations; the rates do not rank the designs.

Backup 12. What Each Number Counts



Mizar 60 (ITP 2023) counts *whole top-level theorems and lemmas*
2,896 holdout problems from 57,897 in
MML 1147; learned premises; 420 CPU s



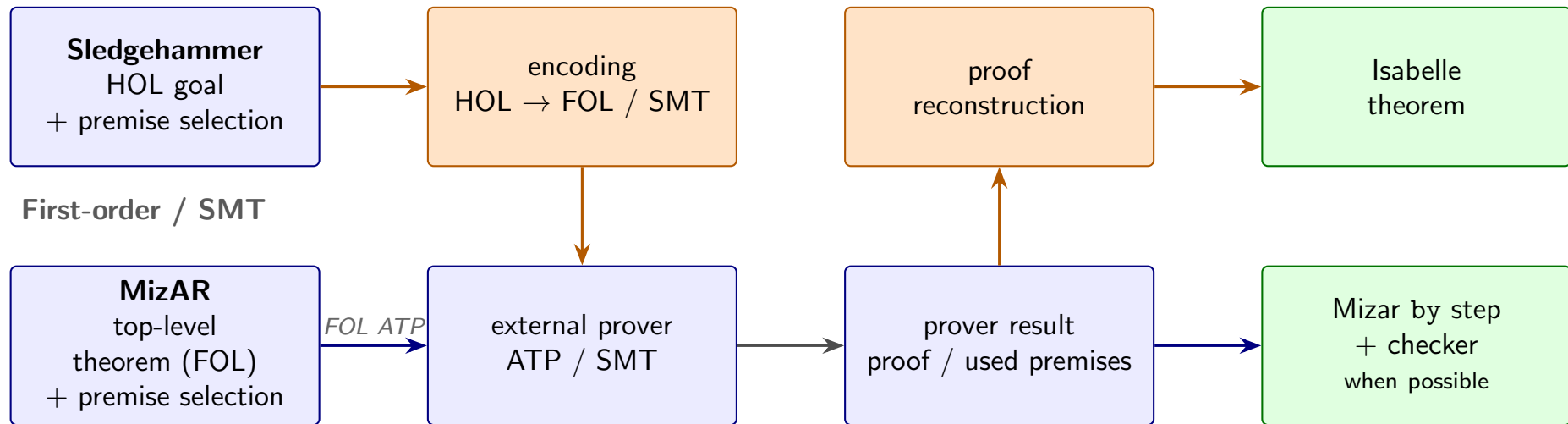
Sledgehammer / AFP (ITP 2022)
counts *proof goals as they arise*
5,000 goals from 50 entries; greedy 16 con-
figurations × 30 CPU s; no reconstruction

Different units, premise selection, and budgets.

- MizAR asks: can the machine prove this whole theorem from the library, with no help from the author?
- The AFP study asks: can the machine close this goal where it appears, often inside a proof that a person has already written?
- Both are fair questions. They are not the same question.

Backup 13. Two Paths To A First-Order Prover

Higher-order (Isabelle/HOL)



Mizar Evo: kernel evidence checking replaces the last step (later slide).

- In Sledgehammer's first-order and SMT path, a higher-order goal is translated, and the proof is rebuilt inside Isabelle.
- MizAR gives the prover a problem that is already first-order. The generated problem is first-order.

Backup 14. Functions In Higher-Order Logic

Schematic HOL statement **sketch** (Isabelle/HOL-style notation)

$f :: 'a \Rightarrow 'b$ $x :: 'a$

$P (f\ x)$

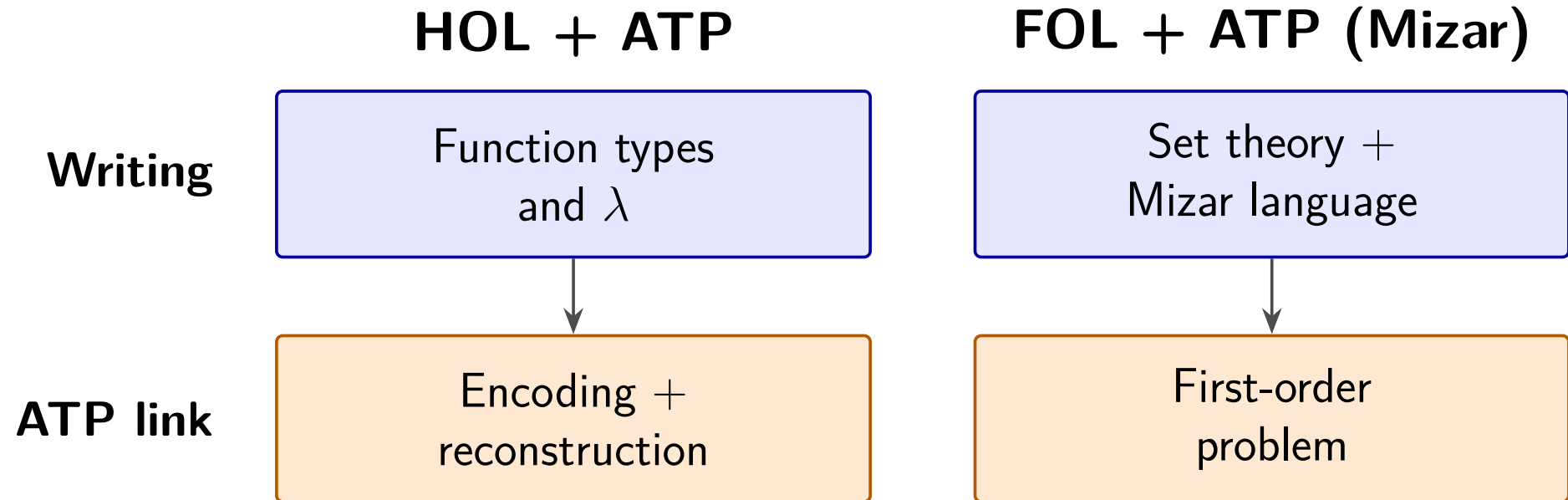
- In HOL, function types and function application are part of the logic itself.
- Functions as arguments, functions that return functions, partial application, lambda: all can be written directly.
- This is very convenient for the person who writes mathematics.
- But E and Vampire are first-order provers. The higher-order structure must be encoded before they can see it.

Backup 15. HOL Encoding And Reconstruction

Encoding steps, schematic **sketch**

<code>F X</code>	<code>-> app(F, X)</code>	
<code>(%x. t) ...</code>	<code>-> fresh constant + defining axioms</code>	<code>(lambda lifting)</code>
<code>polymorphic types</code>	<code>-> type guards or type tags</code>	
<code>Boolean-valued terms</code>	<code>-> extra encoding</code>	

- Applying a variable function becomes an explicit `app` symbol. Lambdas are lifted out or turned into combinators. Types become guards or tags.
- After the prover succeeds, the proof must come back into Isabelle: a `metis` call, an `smt` replay, or generated `Isar` text. This step is called reconstruction.
- These mechanisms connect HOL goals to first-order provers.
- This describes the connection, not a measured performance ranking.



- HOL encodes higher-order structures for ATPs. Mizar's language hides set-theoretic details.
- Mizar Evo keeps the first-order foundation and modernizes mathematical writing.

Backup 17. Design Trade-Offs

	HOL ITP + ATP	FOL ITP + ATP
writing	higher-order features, short text	long text if written raw
functions	basic higher-order objects	first-order set-theoretic objects
ATP connection	HOL-to-FOL / SMT encoding	generation of a first-order problem
reconstruction	reconstruction in Isabelle	rechecking Mizar steps where supported
the language must provide	HOL abstraction	abstraction that hides first-order details

- This table compares representations and checking paths. It gives no measured performance advantage.